RECURSION Solutions

COMPUTER SCIENCE MENTORS 61A

September 30 – October 3, 2025

1 Strings Basics

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> x = 'CS61A '
>>> y = 'CS Mentors'
>>> x + y
```

```
'CS 61A CS Mentors'
```

```
>>> colors = ['red', 'yellow', 'green']
>>> 'green' in colors
```

```
True
```

```
>>> 'GREEN' in colors
```

```
False
```

```
>>> luggage = [c + ' suitcase' for c in colors]
>>> luggage
```

```
['red suitcase', 'yellow suitcase', 'green suitcase']
```

Created by Anaisha Das and Tavisha Patel. Worksheet feedback: forms.gle/q67BZrHHfoKbF6V28
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

2 List Basics

1. What would Python display? Draw box-and-pointer diagrams for the following:

```
>>> a = [1, 2, 3]
>>> a
```

```
[1, 2, 3]
```

```
>>> a[2]
```

```
3
```

```
>>> a[:2]
```

```
[1, 2]
```

```
>>> b = [1, 2, 3, a, 4]
>>> b
```

```
[1, 2, 3, [1, 2, 3], 4]
```

```
>>> b[3][2]
```

```
3
```

2. Write a list comprehension that accomplishes each of the following tasks.

   (a) Square all the elements of a given list, `lst`.

   ```
   [x ** 2 for x in lst]
   ```

   (b) Compute the sum of all even numbers in the list `lst`. Is there a function you can apply to a list to get the sum of all elements?

   ```
   sum([x for x lst1 if x % 2 == 0])
   ```

   (c) Return a list of lists such that `a = [[0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4]]`.

   ```
   a = [[x for x in range(y)] for y in range(1, 6)]
   ```

   (d) Return the same list as above, except now excluding every instance of the number 2: `b = [[0], [0, 1], [0, 1], [0, 1, 3], [0, 1, 3, 4]])`.

   ```
   b = [[x for x in range(y) if x != 2] for y in range(1, 6)]
   ```

3 Dictionaries

1. Using a dictionary comprehension, complete the function below. It takes a single-argument function `f` and a dictionary `d`, and returns a new dictionary containing only the entries whose keys are even numbers, with `f` applied to their values.

```
def apply_to_even(f, d):
    '''
    >>> apply_to_even(lambda x: x**2, {1:3, 2:5, 4:2, 3:2})
    {2: 25, 4: 4}
    '''
    return {__ : ______ for _____ in _____ if ____________}
```

```
def apply_to_even(f, d):
    return {i: f(d[i]) for i in d if i % 2 == 0}
```

4 List Recursion

1. Complete the definition for `all_true`, which takes in a list `lst` and returns `True` if there are no False-y values in the list and `False` otherwise. Make sure that your implementation is recursive. What is the name of the built-in Python function that does this?

```
def all_true(lst):
    '''
    >>> all_true([True, 1, 'True'])
    True
    >>> all_true([1, 0, 1])
    False
    >>> all_true([])
    True
    '''


    if not lst:
        return True
    elif not lst[0]:
        return False
    else:
        return all_true(lst[1:])

    The built-in Python function all does the same thing that all_true
       does!
```

2. Given a list of integers `lst`, return the maximum sum of a subset of size `n`. If `n` is greater than or equal to the length of `lst`, just return the sum of the elements `lst`.

```
def max_subset_sum(lst, n):
    '''
    >>> max_subset_sum([1, 2, 3, 4], 2)
    7
    >>> max_subset_sum([1, 4, 2, 0, 6], 3)
    12
    '''

    if ____________________________:

        ________________________________

    elif ______________________________:

        ________________________________

    with_elem = ____________________________________ + ___________

    without_elem = ____________________________________________

    return ______________________________________________________
```

```
def max_subset_sum(lst, n):
    if n == 0:
        return 0
    elif len(lst) <= n:
        return sum(lst)
    with_elem = max_subset_sum(lst[1:], n - 1) + lst[0]
    without_elem = max_subset_sum(lst[1:], n)
    return max(with_elem, without_elem)
```